Practice · Whole programs 4

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What the report program does

Read the whole program before you answer.

```
#include <stdio.h>

int read_between(int low, int high) {
    int value;
    scanf("%d", &value);
    while (value < low || value > high) {
        printf("Type %d to %d: ", low, high);
        scanf("%d", &value);
    }
    return value;
}

char letter(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'D';
    }
    return 'F';
}

void print_report(int n, int score) {
    printf("Student %d: %d points, ", n, score);
    printf("grade %c\n", letter(score));
}

int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("Score of student %d: ", i);
        int score = read_between(0, 100);
        print_report(i, score);
    }
    return 0;
}
```

What does the whole program do?

- It reads one score and prints three grades for it.
- **Right answer:** For three students, it reads a score from 0 to 100 and prints the score with its letter grade.
- It prints the grades A to F for every score from 0 to 100.

- It reads three scores and prints their average.

Feedback: `main` reads like a list of steps: a loop for three students, and for each one, read a score, then print the report. The work of each step is in its own function. Class 7, slides 64 to 70.

(mc) L1 · Which function checks the score

Same program as the question before. Which function makes sure the score is between 0 and 100?

- `letter`
- `print_report`
- **Right answer:** `read_between`
- `main`

Feedback: `read_between(0, 100)` reads a number and keeps asking with `while` until it is between `low` and `high`. It is class 6's input check, now with a name. Class 7, slides 61 and 67.

(mc) L1 · What letter gives back

In the report program, what does `letter(score)` give back?

- A whole number from 0 to 100.
- Nothing: it prints the grade itself.
- A word such as `"good"`.
- **Right answer:** One `char`: `'A'`, `'B'`, `'C'`, `'D'` or `'F'`.

Feedback: The first line says `char letter(int score)`: it gets an `int` and gives back a `char`. `print_report` prints that char with `%c`. Class 7, slides 59 and 68.

(mc) L1 · What power computes

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 1; i <= exp; i++) {
        result *= base;
    }
    return result;
}
```

What does `power(base, exp)` give back?

- **Right answer:** `base` multiplied by itself `exp` times, for example `power(2, 3)` is 2 × 2 × 2.
- `base` times `exp`, for example `power(2, 3)` is 6.
- `base` plus `exp`.
- `exp` multiplied by itself `base` times.

Feedback: The loop runs `exp` times and multiplies `result` by `base` each time. So `power(2, 3)` is 8 and `power(3, 2)` is 9: the order of the values matters. Class 7, slides 38 to 40.

(mc) L1 · How max3 works

```
int max(int a, int b) {
    if (a > b) {
        return a;
    }
```

```
    return b;
}

int max3(int a, int b, int c) {
    return max(max(a, b), c);
}
```

How does `max3` find the largest of three numbers?

- It compares all three numbers with one `if`.
- **Right answer:** It calls `max` twice: first the larger of `a` and `b`, then the larger of that and `c`.
- It sorts the three numbers.
- It calls itself.

Feedback: A function can call another function. The inner call, `max(a, b)`, is replaced by its value, and that value goes into the outer call with `c`. Class 7, slides 51 and 52.

(mc) L1 · is_even in an if

```
int is_even(int n) {
    if (n % 2 == 0) {
        return 1;
    }
    return 0;
}

int main(void) {
    for (int i = 1; i <= 5; i++) {
        if (is_even(i)) {
            printf("%d is even\n", i);
        } else {
            printf("%d is odd\n", i);
        }
    }
    return 0;
}
```

Why can `is_even(i)` sit straight inside the `if`?

- Because `is_even` prints `even` or `odd` itself.
- Because the `if` calls `is_even` only when `i` is even.
- Because `is_even` is written above `main`.
- **Right answer:** Because it gives back 1 or 0, and in C 1 is true and 0 is false.

Feedback: An `if` needs a value that is true or false. `is_even(i)` is replaced by 1 or 0, so `if (is_even(i))` reads like English: "if i is even". Class 7, slides 53 and 54.

(mc) L1 · Why x stays 5

```
void double_it(int n) {
    n = n * 2;
    printf("inside: n = %d\n", n);
}
```

```
int main(void) {
    int x = 5;
    double_it(x);
    printf("after: x = %d\n", x);
    return 0;
}
```

It prints `inside: n = 10` and then `after: x = 5`. Why is `x` still 5?

- Because `double_it` has a bug in its `printf`.
- Because `n * 2` is not stored anywhere.
- **Right answer:** Because `n` got a copy of 5. The function doubled the copy, not `x`.
- Because `x` is an `int`.

Feedback: At the call, the value 5 is copied into the parameter `n`, a new box. `n` becomes 10 and is gone when the function ends. `main`'s `x` was never touched. This is call-by-value. Class 8, slides 17 to 22.

(mc) L1 · The base case of sum_to

```
int sum_to(int n) {
    if (n == 0) {
        return 0;
    }
    return n + sum_to(n - 1);
}
```

Which part is the **base case**?

- **Right answer:** `if (n == 0) { return 0; }`
- `return n + sum_to(n - 1);`
- `int sum_to(int n)`
- `n - 1`

Feedback: The base case is the small case the function answers directly, with no more calls: the sum of the numbers up to 0 is 0. The line with `sum_to(n - 1)` is the recursive call. Class 8, slides 33, 37 and 52.

(mc) L1 · The step closer in factorial

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

Why is it important that the call is `factorial(n - 1)` and not `factorial(n)`?

- `factorial(n)` would not compile.
- `n - 1` makes the answer smaller.
- It is not important: both give the same answer.
- **Right answer:** Each call gets a smaller `n`, so the calls reach the base case, `n <= 1`, and stop.

Feedback: Every recursive function needs a base case and a step closer to it. With `factorial(n)` the value would never get smaller, the base case would never come, and the program would crash after thousands of calls. Class 8, slides 34, 37 and 40.

(mc) L2 · Three grades

The report program of the first question runs. You type `85`, then `60`, then `59`. What are the three grades?

- `B`, `C`, `D`
- `B`, `D`, `D`
- **Right answer:** `B`, `D`, `F`
- `A`, `D`, `F`

Feedback: `letter` checks from the top: 85 is not >= 90 but is >= 80, so `B`. 60 fails 90, 80 and 70 but passes >= 60, so `D`. 59 fails all four, so the last line gives `F`. The program prints `Student 1: 85 points, grade B`, then `Student 2: 60 points, grade D`, then `Student 3: 59 points, grade F`. Class 7, slides 68 and 70.

(mc) L2 · power(2, 5)

With the `power` function that uses a loop (`result` starts at 1, then `result *= base;` runs `exp` times), what is `power(2, 5)`?

- `10`
- `25`
- `16`
- **Right answer:** `32`

Feedback: The loop runs 5 times: 2, 4, 8, 16, 32. Not 2 × 5, and not 5 × 5. Class 7, slides 38 and 39.

(mc) L2 · max3 with negative numbers

With `max` and `max3` as above, what is `max3(-4, -9, -1)`?

- **Right answer:** `-1`
- `-9`
- `-4`
- `1`

Feedback: `max(-4, -9)` is -4, because -4 is greater than -9. Then `max(-4, -1)` is -1. The largest of three negative numbers is the one closest to zero. Class 7, slides 51 and 52.

(mc) L2 · Two factorials added

```
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main(void) {
    printf("%d\n", factorial(3) + factorial(4));
    return 0;
```

```
}
```

What does it print?

- `7`
- `5040`
- **Right answer:** `30`
- `144`

Feedback: `factorial(3)` is 1 × 2 × 3 = 6 and `factorial(4)` is 6 × 4 = 24. Each call is replaced by its value, so the line adds 6 + 24 = 30. Class 7, slide 56.

(mc) L2 · double_it, then the fix

```
void double_it(int n) {
    n = n * 2;
}

int double_ret(int n) {
    return n * 2;
}

int main(void) {
    int x = 7;
    double_it(x);
    printf("%d ", x);
    x = double_ret(x);
    printf("%d\n", x);
    return 0;
}
```

What does it print?

- `14 28`
- **Right answer:** `7 14`
- `14 14`
- `7 7`

Feedback: `double_it` doubles its own copy, so `x` is still 7. `double_ret` returns 14 and `x = double_ret(x);` stores it. Returning the new value and keeping it is the fix. Class 8, slides 19 and 25.

(mc) L2 · swap with 5 and 8

```
void swap(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("in swap: %d %d\n", a, b);
}

int main(void) {
    int p = 5, q = 8;
    swap(p, q);
```

```
    printf("in main: %d %d\n", p, q);
    return 0;
}
```

The first line is `in swap: 8 5`. What is the second line?

- `in main: 8 5`
- `in main: 8 8`
- `in main: 5 5`
- **Right answer:** `in main: 5 8`

Feedback: `swap` swapped its copies, `a` and `b`. `main`'s `p` and `q` were never touched. Changing the caller's variables needs pointers, class 18. Class 8, slides 26 and 27.

(mc) L2 · sum_to(5)

With the recursive `sum_to` above (base case: `sum_to(0)` is 0; step: `n + sum_to(n - 1)`), what is `sum_to(5)`?

- `5`
- **Right answer:** `15`
- `10`
- `120`

Feedback: Down: `sum_to(5)` needs `sum_to(4)`, ... down to `sum_to(0)`, which is 0. Back up: 1, 3, 6, 10, 15. It adds, it does not multiply, so it is not 120. Class 8, slides 51 and 52.

(mc) L2 · Recursive power

```
int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    }
    return base * power(base, exp - 1);
}
```

What is `power(2, 6)`?

- `12`
- `32`
- **Right answer:** `64`
- `128`

Feedback: Down: `power(2, 6)` needs `power(2, 5)`, ... down to `power(2, 0)`, which is 1. Back up, each call multiplies by 2: 2, 4, 8, 16, 32, 64. Class 8, slide 53.

(mc) L2 · count_up(4)

```
void count_up(int n) {
    if (n == 0) {
        return;
    }
    count_up(n - 1);
    printf("%d\n", n);
```

```
}
```

What does `count_up(4)` print, one number per line?

- `4 3 2 1`
- `0 1 2 3 4`
- `4`
- **Right answer:** `1 2 3 4`

Feedback: The `printf` comes after the recursive call. So each call waits until the smaller calls have finished, and the numbers are printed on the way back up: 1 first, 4 last. `count_up(0)` returns before its `printf`, so 0 is not printed. Class 8, slides 47 and 48.

(mc) L2 · Find the bug: every good score is D

Someone changed the order inside `letter`:

```
char letter(int score) {
    if (score >= 60) {
        return 'D';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 90) {
        return 'A';
    }
    return 'F';
}
```

Now 95 and 72 both get `D`, and 40 gets `F`. What is the bug?

- `return 'F';` must be inside an `else`.
- **Right answer:** The checks are in the wrong order: 95 is already `>= 60`, so the first `return` ends the function before the other checks.
- A `char` function cannot have five `return` lines.
- `score` must be a `double`.

Feedback: `return` ends the function at once, and the first true check wins. Every score of 60 or more stops at the first `if`. Check the highest grade first: `>= 90`, then 80, 70, 60, as in class 7, slide 59. Class 7, slides 49 and 59.

(fib) L3 · Complete read_between: the condition

The loop must ask again while the value is **outside** `low` to `high`. The first half of the condition is there. Complete the second half: the value is too big. Type it with single spaces, like `a > b`.

```
int read_between(int low, int high) {
    int value;
    scanf("%d", &value);
    while (value < low || [value > high|value>high|high < value|high<value]) {
        printf("Type %d to %d: ", low, high);
        scanf("%d", &value);
```

```
    }
    return value;
}
```

Feedback: The condition describes the **bad** input: below `low` or above `high`. When you type 101 for `read_between(0, 100)`, `101 > 100` is true, so it prints `Type 0 to 100:` and reads again. Class 7, slides 61 and 71.

(fib) L3 · Complete print_report: the call

Complete the `printf` so it prints the letter of `score`. Type only the call, the piece after the comma.

```
void print_report(int n, int score) {
    printf("Student %d: %d points, ", n, score);
    printf("grade %c\n", [letter(score)|letter( score )]);
}
```

Feedback: A function can call another function. `letter(score)` is replaced by a `char` such as `'B'`, and `%c` prints it. Class 7, slide 69.

(fib) L3 · Complete power: the operator

Type the operator that makes `result` get multiplied by `base` every round.

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 1; i <= exp; i++) {
        result [*=] base;
    }
    return result;
}
```

Feedback: `result *= base;` means `result = result * base;`. With `+=` you would get 1 + 2 + 2 + ... instead of a power. Class 7, slides 39 and 40.

(fib) L3 · Complete max3: the inner call

Complete `max3` so it gives back the largest of `a`, `b` and `c`. Type the inner call, the first value inside the outer `max( , c)`.

```
int max(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}

int max3(int a, int b, int c) {
    return max([max(a, b)|max(a,b)|max(b, a)|max(b,a)], c);
}
```

Feedback: First the larger of `a` and `b`, then the larger of that and `c`. `max3(4, 11, 7)` is 11. Class 7, slides 51 and 52.

(fib) L3 · Complete is_even: the condition

Complete the condition that is true when `n` is even. Type it with single spaces, like `n % 3 == 1`.

```
int is_even(int n) {
    if ([n % 2 == 0|n%2 == 0|n % 2==0|n%2==0]) {
        return 1;
    }
    return 0;
}
```

Feedback: A number is even when dividing it by 2 leaves nothing: `n % 2 == 0`. Then the function gives back 1 for yes. Class 7, slide 53.

(fib) L3 · Complete the fix: keep the doubled value

Complete the line so that `x` becomes 10. Type only what comes after `x =`, without the semicolon.

```
int double_it(int n) {
    return n * 2;
}

int main(void) {
    int x = 5;
    x = [double_it(x)|double_it( x )];
    printf("after: x = %d\n", x);
    return 0;
}
```

Feedback: `double_it(x)` gives back 10, and `x =` stores it. The program prints `after: x = 10`. Class 8, slide 25.

(fib) L3 · Complete factorial: the step closer

Complete the recursive call so that each call gets a smaller number. Type what goes inside the brackets, like `n - 2`.

```
int factorial(int n) {
    if (n <= 1) {
        return 1;
    }
    return n * factorial([n - 1|n-1]);
}
```

Feedback: n! = n × (n - 1)!, so the call is `factorial(n - 1)`. Each call is one smaller, until `n <= 1`. `factorial(4)` gives back 24. Class 8, slides 40 to 44.

(fib) L3 · Complete sum_to: the base case

Complete the base case: the sum of the numbers up to 0. Type one number.

```
int sum_to(int n) {
    if (n == 0) {
        return [0];
    }
    return n + sum_to(n - 1);
}
```

Feedback: The sum of nothing is 0. With 0 there, `sum_to(4)` is 4 + 3 + 2 + 1 + 0 = 10. With 1 there, every answer would be one too big. Class 8, slides 51 and 52.

(fib) L3 · Complete power: the step closer

Complete the recursive call of `power`. Type the second value inside the brackets, like `exp - 2`.

```
int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    }
    return base * power(base, [exp - 1|exp-1]);
}
```

Feedback: Each call multiplies by `base` once and asks for one power less, until `exp` is 0 and the answer is 1. `power(2, 10)` gives back 1024. Class 8, slide 53.